```
d3.select("circle").node()
<circle r="20" style="fill: #ffc0cb; stroke: #000000; stroke-width: 1px;"></circle>
```

Figure 3.8 The results of running the `node` function of a selection in the console, which is the DOM element itself—in this case, an SVG `<circle>` element.

Getting to the DOM element, as shown in figure 3.8, lets you take advantage of built-in JavaScript functionality to do things like measure the length of a `<path>` element or clone an element. One of the most useful built-in functions of nodes when working with SVG is the ability to re-append a child element. Remember that SVG has no Z-levels, which means that the drawing order of elements is determined by their DOM order. Drawing order is important because you don't want the graphical objects you interact with to look like they're behind the objects that you don't interact with. To see what this means, let's first adjust our highlighting function so that it increases the size of the label when we mouse over each element:

```
function highlightRegion2(d,i) {
    d3.select(this).select("text").classed("active", true).attr("y", 10);
    d3.selectAll("g.overallG").select("circle").each(function(p,i) {
     p.region == d.region ?
        d3.select(this).classed("active",true) :
        d3.select(this).classed("inactive",true);
   });
};
```

By turning on "active" class for the `<g>` that we hover over, we take advantage of the "g > text.active" rule in CSS that makes any text elements in that `<g>` increase their font size.

Because we're doing a bit more, we should change the `mouseout` event to point to a function, which we'll call `unHighlight`:

```
teamG.on("mouseout", unHighlight)

function unHighlight() {
 d3.selectAll("g.overallG").select("circle").attr("class", "");
 d3.selectAll("g.overallG").select("text")
.classed("highlight", false).attr("y", 30);
};
```

As shown in figure 3.9, Germany was appended to the DOM before Argentina. As a result, when we increase the size of the graphics associated with Germany, those graphics remain behind any graphics for Argentina, creating a visual artifact that looks unfinished and distracting. We can rectify this by re-appending the node to the parent `<g>` during that same highlighting event, which results in the label being displayed above the other elements, as shown in figure 3.10:



Figure 3.9 The `<text>` element "Germany" is drawn at the same DOM level as the parent `<g>`, which, in this case, is behind the element to its right.